# TKO_7092 Evaluation of Machine Learning Methods 2026

## Exercise 3

---

Student name: Talha Rizwan

Student number: 2509714

Student email: tarizw@utu.fi

---

## IMPORTANT

This exercise involves using AI. Use only the *Study Chat* service provided by the University of Turku at https://ai.utu.fi/en. **Do not use any other AI service!** Before starting, remember to carefully read the guidelines for using AI (https://intranet.utu.fi/en/sites/ai/guidelines/Pages/default.aspx) and the terms of use. **Do not share any personal information or copyrighted material with AI.**

Save all your discussions (including your prompts and AI's output) as well as the name of the model you used.

## Instructions

The deadline of this exercise is **Wednesday 18 February 2026 at 11:59 PM**. Please contact Juho Heimonen (juaheim@utu.fi) if you have any questions about the exercise. Remember to follow all the general exercise guidelines that are stated in Moodle.

The exercise has several parts, all of which concern the letter below. You will take the role of a data scientist who has been assigned to solve the problem described in the letter. You have an AI tool to assist you, but you alone are responsible for the quality of the solution.

### 1

Ask AI to write code to solve the task. Analyse which parts of the AI-generated code are correct and which are incorrect. Pay particular attention to the key parts of the cross-validation. You may ask AI to improve the code as many times as you like, as long as you keep analysing its output.

### 2

Implement the required leave-one-out cross-validations and run your code to get the estimates you were asked to obtain. Here it is okay to use any amount of the AI-generated code you produced above. You can use a complete, fully correct AI-written solution, you can write the implementation from scratch by yourself, or you can take some AI-generated code and complete the implementation manually.

## 3

Write a report in which you discuss the following:

- Why did the cross-validation described in the letter fail? What is the correct way to do cross-validation here?
- Which parts of the task was AI able to code correctly and which not? Focus particularly on the core of the cross-validation.
- Which parts of the AI-generated code did you use in your implementation? Why? Explain why the selected pieces of code work correctly in your implementation.
- What results did you get with your implementation? Report the estimates and interpret the results in terms of how well the model will work in the situations described in the letter. Explain in detail why your cross-validation is the correct way to estimate the generalisation performance.

Write the report in your own words and explain everything clearly, precisely, and comprehensively. **You are not allowed to use AI to write the report for you** because this is where you show that you have understood the theory and are able to apply it. If you use AI as a teacher (i.e. to explain things to you for learning purposes), you must attach the discussions and clearly state what and how you learnt from the AI. **If there is uncertainty about how the text was produced, you may be required to explain the content of your report in a face-to-face meeting.**

## 4

Submit the following documents to Moodle:

- The discussions with AI (including your prompts and AI's output), as PDF.
- The implementation of your cross-validation, as PDF and as a Jupyter notebook.
- The report, as PDF. (It is okay to integrate the report to the Jupyter notebook.)

Note that it is not enough to just implement the cross-validation correctly to pass this exercise. You must also explain in plain words what you have done and demonstrate that you understand how cross-validation should be performed on pair-input data. Small errors are acceptable, but you will fail this exercise if there are significant error(s) or omission(s) in the report or in the implementation.

# Letter from your client

Dear Data Scientist,

I have a long-term research project regarding a specific set of proteins. I am attempting to discover small organic compounds that can bind strongly to these proteins and thus act as drugs. I have already made laboratory experiments to measure the affinities between some proteins and drug molecules.

My colleague is working on another set of proteins, and the objectives of his project are similar to mine. He has recently discovered thousands of new potential drug molecules. He asked me if I could find the pairs that have the strongest affinities among his proteins and drug molecules. Obviously I do not have the resources to measure all the possible pairs in my laboratory, so I need to prioritise. I decided to do this with the help of machine learning, but I have encountered a problem.

Here is what I have done so far: First I trained a K-nearest neighbours regressor with the parameter value K=10 using all the 400 measurements I had already made in the laboratory with my proteins and drug molecules. They comprise of 77 target proteins and 59 drug molecules. To estimate the generalisation performance of the model, I then performed a leave-one-out cross-validation. I used C-index and got a stellar score above 90%. Finally I used the model to predict the affinities of my colleague's proteins and drug molecules. The problem is that when I selected the highest predicted affinities and my colleague tried to verify them in the lab, we found that many of them are much lower in reality. My model clearly does not work despite the high cross-validation score. We also tested the model with my proteins against my colleague's drugs (which is another task I would like to use my model for), but the model did not work there either.

Please explain why my estimation failed and how leave-one-out cross-validation should be performed to get reliable estimates. Also, please implement the leave-one-out cross-validation correctly and report the numbers I need. I want to know whether it would be a waste of my colleague's and my resources if we were to use my model any further.

The data I used to create my model is available in the files `input.data`, `output.data` and `pairs.data` for you to use. The first file contains the features of the pairs, whereas the second contains their affinities. The third file contains the identifiers of the drug and target molecules of which the pairs are composed. The files are paired, i.e. the $i^{th}$ row in each file is about the same pair.

Looking forward to hearing from you soon.

Yours sincerely,
Bio Scientist

# Part 1: AI-generated baseline code (Mistral Devstral 2)

Below is the baseline implementation from Mistral Dev.

In [5]:
```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import LeaveOneOut
import matplotlib.pyplot as plt

# Custom function to calculate C-index
def c_index(y_true, y_pred):
    """
    Calculate the concordance index (C-index) for evaluating prediction performance

    Parameters:
    y_true (array-like): True target values
    y_pred (array-like): Predicted target values

    Returns:
    float: C-index score (between 0 and 1, higher is better)
    """
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    concordant = 0
    permissible = 0

    for i in range(len(y_true)):
        for j in range(i + 1, len(y_true)):
            if y_true[i] != y_true[j]:
                permissible += 1
                if (y_pred[i] > y_pred[j] and y_true[i] > y_true[j]) or \
                   (y_pred[i] < y_pred[j] and y_true[i] < y_true[j]):
                    concordant += 1

    if permissible == 0:
        return 0.0

    return concordant / permissible
```

In [6]:
```
# 1. Load and preprocess the data
def load_and_preprocess_data(input_file, output_file, pairs_file):
    """
    Load and preprocess the dataset.

    Parameters:
    input_file (str): Path to input features file
    output_file (str): Path to output affinities file
    pairs_file (str): Path to pairs identifiers file

    Returns:
```

```
    tuple: (X, y, pairs_df) where X is features, y is targets, pairs_df contains id
    """
    X = pd.read_csv(input_file, header=None, sep=r"\s+").values
    y = pd.read_csv(output_file, header=None, sep=r"\s+").values.ravel()
    pairs_df = pd.read_csv(
        pairs_file,
        header=None,
        names=['drug_id', 'protein_id'],
        sep=r"\s+",
        quotechar='"'
    )

    X_mean = X.mean(axis=0)
    X_std = X.std(axis=0)
    X_std[X_std == 0] = 1.0
    X = (X - X_mean) / X_std

    return X, y, pairs_df
```

In [7]:

```
# 2. Train KNN regressor with K=10 and perform LOOCV
def knn_loocv_with_cindex(X, y, k=10):
    """
    Perform leave-one-out cross-validation with KNN regressor and evaluate using C-

    Parameters:
    X (array-like): Input features
    y (array-like): Target values
    k (int): Number of neighbors for KNN

    Returns:
    tuple: (predictions, c_index_score)
    """
    knn = KNeighborsRegressor(n_neighbors=k)
    loo = LeaveOneOut()

    predictions = np.zeros_like(y)

    for train_index, test_index in loo.split(X):
        X_train, X_test = X[train_index], X[test_index]
        y_train = y[train_index]

        knn.fit(X_train, y_train)
        predictions[test_index] = knn.predict(X_test)

    c_index_score = c_index(y, predictions)
    return predictions, c_index_score
```

In [8]:

```
# 3. Visualization functions
def plot_true_vs_predicted(y_true, y_pred):
    """
    Plot true vs predicted values.
    """
    plt.figure(figsize=(8, 6))
    plt.scatter(y_true, y_pred, alpha=0.5)
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--', lw=
```

```
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title('True vs Predicted Affinity Values')
    plt.grid(True)
    plt.show()


def plot_residuals(y_true, y_pred):
    """
    Plot residuals (errors) of predictions.
    """
    residuals = y_true - y_pred
    plt.figure(figsize=(8, 6))
    plt.scatter(y_pred, residuals, alpha=0.5)
    plt.axhline(y=0, color='r', linestyle='-')
    plt.xlabel('Predicted Affinity')
    plt.ylabel('Residuals (True - Predicted)')
    plt.title('Residual Plot')
    plt.grid(True)
    plt.show()
```

In [9]:

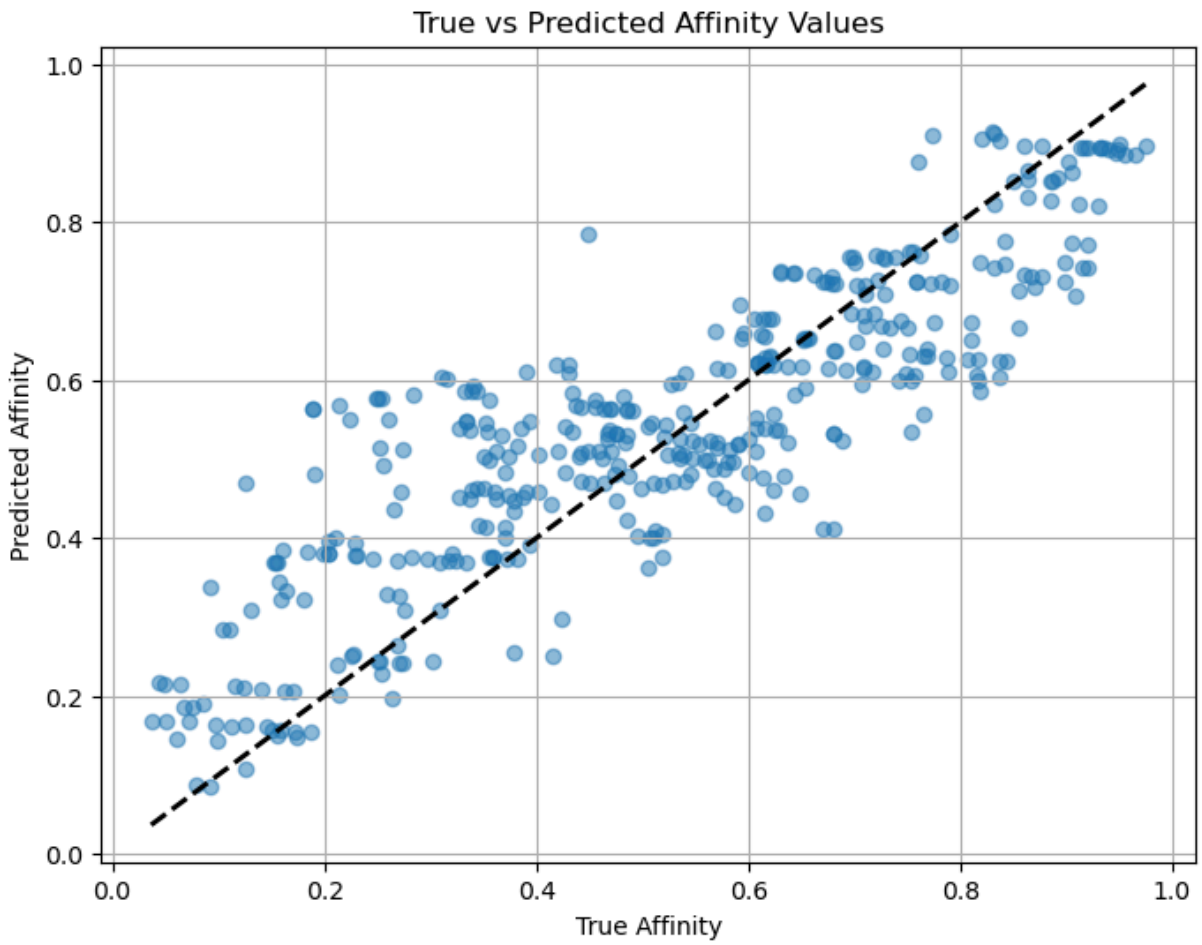
```
# 4. Main execution
input_file = 'input.data'
output_file = 'output.data'
pairs_file = 'pairs.data'

X, y, pairs_df = load_and_preprocess_data(input_file, output_file, pairs_file)
print(f"Data loaded. Features shape: {X.shape}, Targets shape: {y.shape}")

predictions, c_index_score = knn_loocv_with_cindex(X, y, k=10)
print(f"C-index score: {c_index_score:.4f}")
```

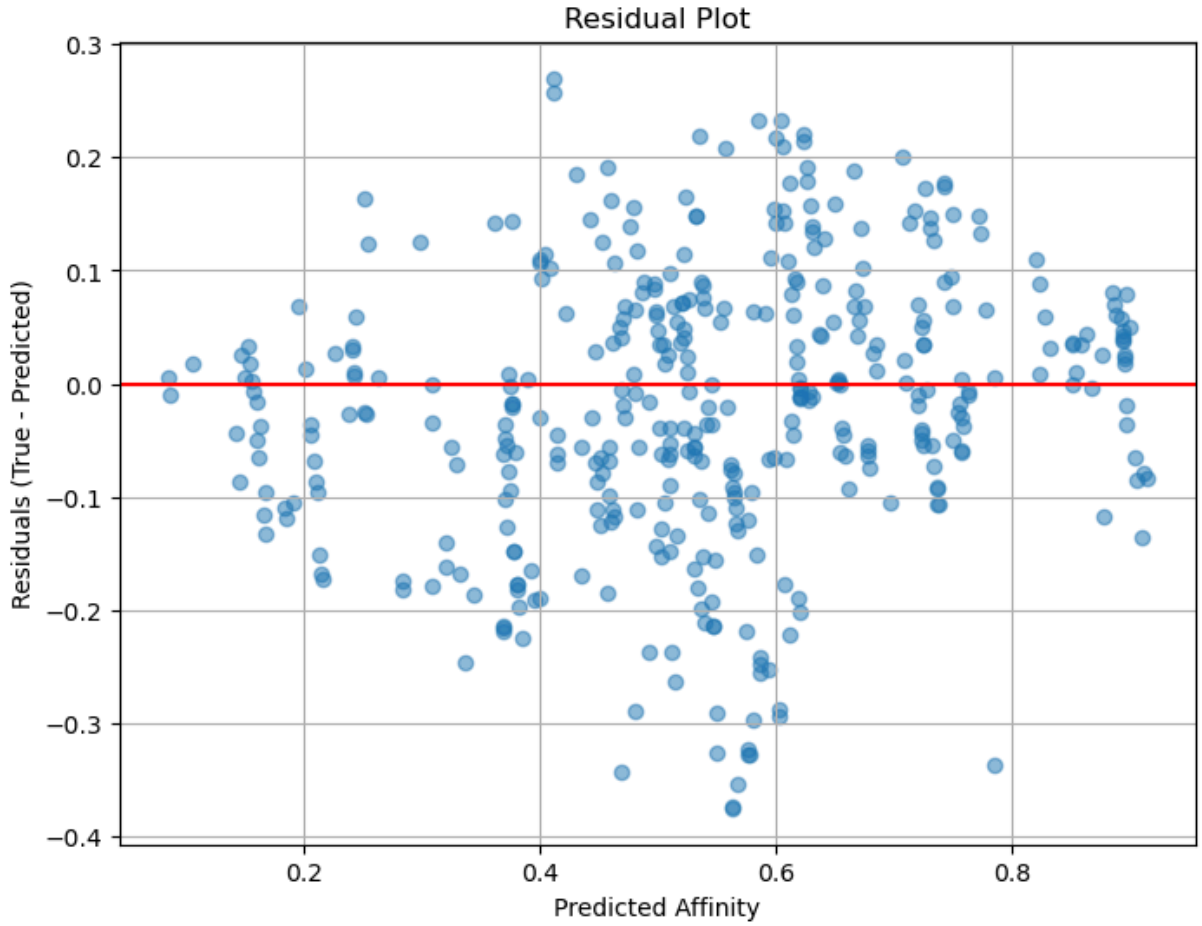
```
Data loaded. Features shape: (400, 67), Targets shape: (400,)
C-index score: 0.8293
```

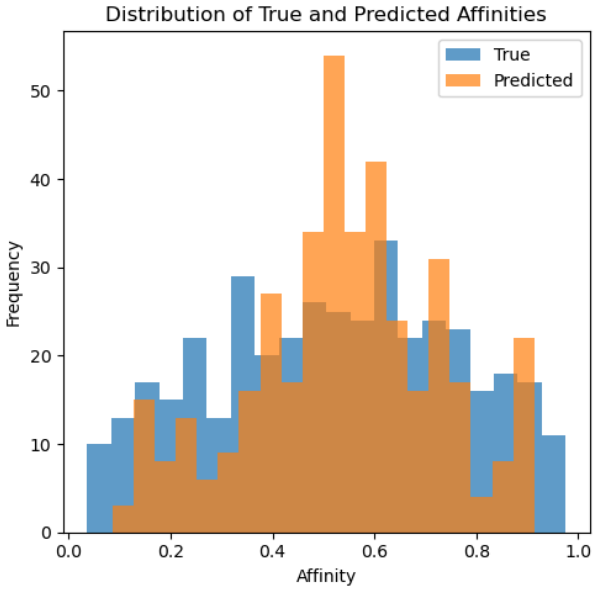
In [10]:

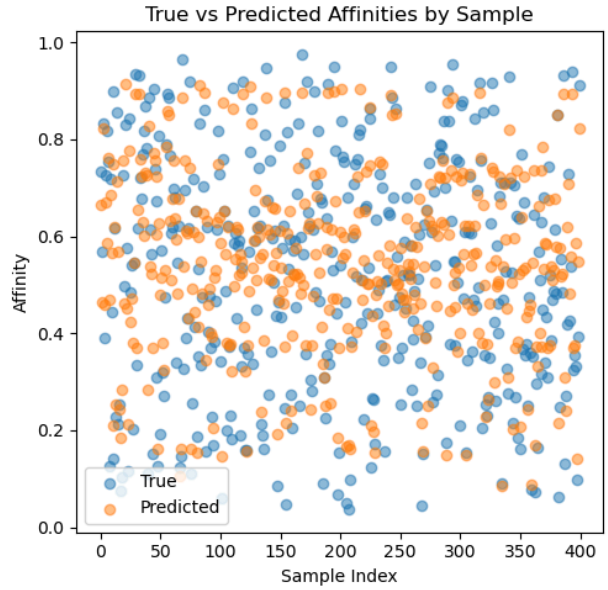
```
# 5. Plot results
plot_true_vs_predicted(y, predictions)
plot_residuals(y, predictions)

# Additional analysis: Distribution of predictions
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.hist(y, bins=20, alpha=0.7, label='True')
plt.hist(predictions, bins=20, alpha=0.7, label='Predicted')
plt.xlabel('Affinity')
plt.ylabel('Frequency')
plt.title('Distribution of True and Predicted Affinities')
plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(range(len(y)), y, label='True', alpha=0.5)
plt.scatter(range(len(predictions)), predictions, label='Predicted', alpha=0.5)
plt.xlabel('Sample Index')
plt.ylabel('Affinity')
plt.title('True vs Predicted Affinities by Sample')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```






## Part 1 analysis of AI-generated code

The AI-generated code is partly correct and useful as a baseline: it correctly loads the three files, trains a KNN regressor with K=10, performs leave one out prediction over pairs, computes a C index like ranking score, and visualizes predictions and residuals clearly. However, the core validation logic is not correct for this assignment's real deployment setting, because pair level leave one out allows the same drug or protein identities to appear

in both training and test folds, which can produce overly optimistic performance estimates for new-molecule scenarios.

## Part 2: AI-generated corrected validation code (Mistral)

In [11]:
```
import numpy as np
import pandas as pd
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import matplotlib.pyplot as plt
from collections import defaultdict

# Custom C-index function
def c_index(y_true, y_pred):
    y_true = np.asarray(y_true)
    y_pred = np.asarray(y_pred)

    concordant = 0
    permissible = 0

    for i in range(len(y_true)):
        for j in range(i+1, len(y_true)):
            if y_true[i] != y_true[j]:
                permissible += 1
                if (y_pred[i] > y_pred[j] and y_true[i] > y_true[j]) or \
                   (y_pred[i] < y_pred[j] and y_true[i] < y_true[j]):
                    concordant += 1

    return concordant / permissible if permissible > 0 else 0.0
```

In [12]:
```
# Load and preprocess data
def load_data(input_file, output_file, pairs_file):
    X = pd.read_csv(input_file, header=None).values
    y = pd.read_csv(output_file, header=None).values.ravel()
    pairs_df = pd.read_csv(pairs_file, header=None, names=['protein_id', 'drug_id']

    return X, y, pairs_df
```

In [13]:
```
# Leave-One-Drug-Out Cross-Validation
def lodo_cv(X, y, pairs_df, k=10):
    """
    Leave-One-Drug-Out Cross-Validation
    Each fold leaves out all pairs involving one drug
    """
    # Get unique drugs and create mapping
    unique_drugs = pairs_df['drug_id'].unique()
    drug_to_indices = defaultdict(list)

    for idx, drug_id in enumerate(pairs_df['drug_id']):
        drug_to_indices[drug_id].append(idx)
```

```python
    # Initialize predictions array
    predictions = np.zeros_like(y)

    for drug_id in unique_drugs:
        # Get indices for current drug (test set)
        test_indices = drug_to_indices[drug_id]

        # Get training set (all other drugs)
        train_indices = [i for i in range(len(y)) if i not in test_indices]

        X_train, X_test = X[train_indices], X[test_indices]
        y_train = y[train_indices]

        # Create pipeline with scaling and KNN
        model = make_pipeline(
            StandardScaler(),
            KNeighborsRegressor(n_neighbors=k)
        )

        # Train and predict
        model.fit(X_train, y_train)
        predictions[test_indices] = model.predict(X_test)

    return predictions

# Leave-One-Protein-Out Cross-Validation
def lopo_cv(X, y, pairs_df, k=10):
    """
    Leave-One-Protein-Out Cross-Validation
    Each fold leaves out all pairs involving one protein
    """
    # Get unique proteins and create mapping
    unique_proteins = pairs_df['protein_id'].unique()
    protein_to_indices = defaultdict(list)

    for idx, protein_id in enumerate(pairs_df['protein_id']):
        protein_to_indices[protein_id].append(idx)

    # Initialize predictions array
    predictions = np.zeros_like(y)

    for protein_id in unique_proteins:
        # Get indices for current protein (test set)
        test_indices = protein_to_indices[protein_id]

        # Get training set (all other proteins)
        train_indices = [i for i in range(len(y)) if i not in test_indices]

        X_train, X_test = X[train_indices], X[test_indices]
        y_train = y[train_indices]

        # Create pipeline with scaling and KNN
        model = make_pipeline(
            StandardScaler(),
            KNeighborsRegressor(n_neighbors=k)
        )
```

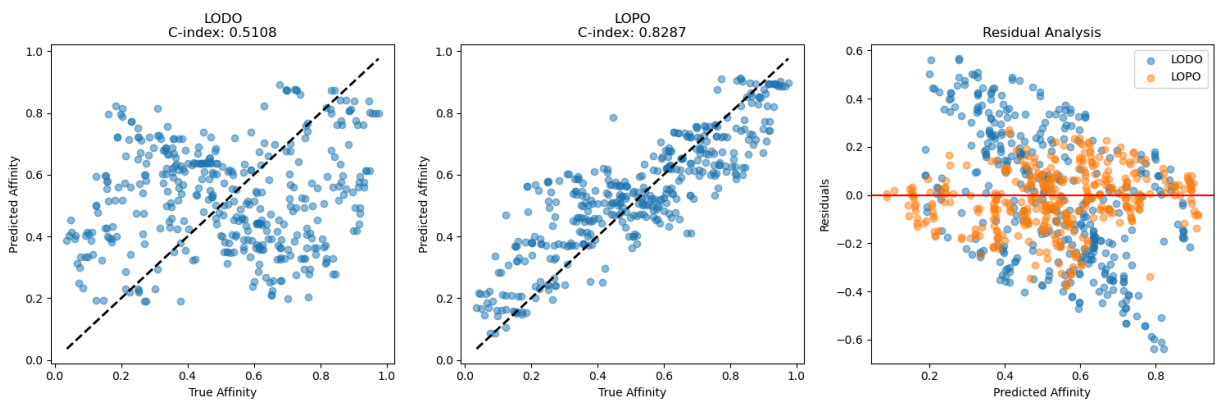
```
        # Train and predict
        model.fit(X_train, y_train)
        predictions[test_indices] = model.predict(X_test)

    return predictions
```

In [14]:
```
# Visualization functions
def plot_results(y_true, y_pred_lodo, y_pred_lopo):
    plt.figure(figsize=(15, 5))

    # LODO results
    plt.subplot(1, 3, 1)
    plt.scatter(y_true, y_pred_lodo, alpha=0.5)
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--', lw=
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title(f'LODO\nC-index: {c_index(y_true, y_pred_lodo):.4f}')

    # LOPO results
    plt.subplot(1, 3, 2)
    plt.scatter(y_true, y_pred_lopo, alpha=0.5)
    plt.plot([y_true.min(), y_true.max()], [y_true.min(), y_true.max()], 'k--', lw=
    plt.xlabel('True Affinity')
    plt.ylabel('Predicted Affinity')
    plt.title(f'LOPO\nC-index: {c_index(y_true, y_pred_lopo):.4f}')

    # Combined residuals
    plt.subplot(1, 3, 3)
    residuals_lodo = y_true - y_pred_lodo
    residuals_lopo = y_true - y_pred_lopo
    plt.scatter(y_pred_lodo, residuals_lodo, alpha=0.5, label='LODO')
    plt.scatter(y_pred_lopo, residuals_lopo, alpha=0.5, label='LOPO')
    plt.axhline(y=0, color='r', linestyle='-')
    plt.xlabel('Predicted Affinity')
    plt.ylabel('Residuals')
    plt.title('Residual Analysis')
    plt.legend()

    plt.tight_layout()
    plt.show()
```

In [16]:
```
# Runtime parsing fix for .data files in this assignment format
# (keeps the rest of the AI-generated Part 2 logic unchanged)
def load_data(input_file, output_file, pairs_file):
    X = pd.read_csv(input_file, header=None, sep=r"\s+").values
    y = pd.read_csv(output_file, header=None, sep=r"\s+").values.ravel()
    pairs_df = pd.read_csv(
        pairs_file,
        header=None,
        names=['drug_id', 'protein_id'],
        sep=r"\s+",
        quotechar='"'
    )
    return X, y, pairs_df
```

In [17]:
```python
# Main execution
input_file = 'input.data'
output_file = 'output.data'
pairs_file = 'pairs.data'

X2, y2, pairs_df2 = load_data(input_file, output_file, pairs_file)
print(f"Data loaded. Features shape: {X2.shape}, Targets shape: {y2.shape}")
print(f"Unique proteins: {pairs_df2['protein_id'].nunique()}")
print(f"Unique drugs: {pairs_df2['drug_id'].nunique()}")

print("\nRunning Leave-One-Drug-Out Cross-Validation...")
lodo_predictions = lodo_cv(X2, y2, pairs_df2, k=10)
lodo_cindex = c_index(y2, lodo_predictions)
print(f"LODO C-index: {lodo_cindex:.4f}")

print("\nRunning Leave-One-Protein-Out Cross-Validation...")
lopo_predictions = lopo_cv(X2, y2, pairs_df2, k=10)
lopo_cindex = c_index(y2, lopo_predictions)
print(f"LOPO C-index: {lopo_cindex:.4f}")

plot_results(y2, lodo_predictions, lopo_predictions)
```

```
Data loaded. Features shape: (400, 67), Targets shape: (400,)
Unique proteins: 77
Unique drugs: 59

Running Leave-One-Drug-Out Cross-Validation...
LODO C-index: 0.5108

Running Leave-One-Protein-Out Cross-Validation...
LOPO C-index: 0.8287
```




In [18]:
```python
# Additional analysis: Performance by protein/drug
# For LODO - performance by drug
drug_performance = defaultdict(list)
for idx, drug_id in enumerate(pairs_df2['drug_id']):
    drug_performance[drug_id].append((y2[idx], lodo_predictions[idx]))

print("\nLODO Performance by Drug (top 5 best and worst):")
drug_scores = {drug: c_index([true for true, _ in pairs],
                             [pred for _, pred in pairs])
               for drug, pairs in drug_performance.items()}
sorted_drugs = sorted(drug_scores.items(), key=lambda x: x[1], reverse=True)
for drug, score in sorted_drugs[:5] + sorted_drugs[-5:]:
```

```
    print(f"Drug {drug}: C-index = {score:.4f}")

# For LOPO - performance by protein
protein_performance = defaultdict(list)
for idx, protein_id in enumerate(pairs_df2['protein_id']):
    protein_performance[protein_id].append((y2[idx], lopo_predictions[idx]))

print("\nLOPO Performance by Protein (top 5 best and worst):")
protein_scores = {prot: c_index([true for true, _ in pairs],
                                [pred for _, pred in pairs])
                  for prot, pairs in protein_performance.items()}
sorted_proteins = sorted(protein_scores.items(), key=lambda x: x[1], reverse=True)
for prot, score in sorted_proteins[:5] + sorted_proteins[-5:]:
    print(f"Protein {prot}: C-index = {score:.4f}")
```

```
LODO Performance by Drug (top 5 best and worst):
Drug D28: C-index = 0.8667
Drug D5: C-index = 0.8000
Drug D33: C-index = 0.8000
Drug D45: C-index = 0.7778
Drug D27: C-index = 0.7619
Drug D19: C-index = 0.0000
Drug D18: C-index = 0.0000
Drug D47: C-index = 0.0000
Drug D36: C-index = 0.0000
Drug D51: C-index = 0.0000

LOPO Performance by Protein (top 5 best and worst):
Protein T49: C-index = 1.0000
Protein T52: C-index = 1.0000
Protein T57: C-index = 1.0000
Protein T32: C-index = 1.0000
Protein T9: C-index = 1.0000
Protein T26: C-index = 0.0000
Protein T68: C-index = 0.0000
Protein T17: C-index = 0.0000
Protein T62: C-index = 0.0000
Protein T37: C-index = 0.0000
```

## Part 2 analysis of AI-generated code

This AI-generated Part 2 implementation improves the core validation idea compared with Part 1 by using leave one drug out (LODO) and leave one protein out (LOPO), and by placing scaling inside a per fold pipeline, which is directionally correct for reducing leakage. It also keeps the requested KNN setting (K=10), produces out of fold predictions, reports overall C-index values, and includes visual diagnostics. However, there are still important weaknesses to note, the loading of pairs data assumes a plain CSV layout, while this file contains quoted, whitespace-separated identifiers in this dataset.